

Imperial College London

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Trajectory-Level Refactoring Analysis and Actionable Explanations Using IDE Traces

Author:
Ethan Hosier

Supervisor:
Dr Robert Chatley

Second Marker:

-

January 21, 2026

Abstract

Your abstract goes here

Acknowledgements

Contents

1	Introduction	5
1.1	Background	5
1.2	Motivation	5
1.3	Research Objectives	6
1.4	Approach and Scope	6
2	Background and Related Work	8
2.1	Definitions and framing	8
2.1.1	Refactoring as behaviour-preserving transformation	8
2.1.2	Atomic refactorings, smells, and sequences	8
2.1.3	State-action-trace formalism	9
2.2	Outcome-based refactoring evaluation	9
2.2.1	Endpoint-centric quality improvement	9
2.2.2	What endpoint evaluation captures—and what it misses	9
2.3	Code smell detection and quality metrics	10
2.3.1	Smell detection as operationalising design issues	10
2.3.2	Quality metrics: structural and readability-oriented signals	10
2.3.3	Implications for process-quality metric construction	10
2.4	Refactoring detection and mining from code changes	11
2.4.1	From diffs to refactoring actions	11
2.4.2	Representation, coverage, and typical failure modes	12
2.4.3	Tie-in to this thesis	12
2.5	Refactoring recommendation and search/synthesis	12
2.5.1	Synthesis and path completion from partial edits	13
2.5.2	Search-based optimisation of refactoring sequences	13
2.5.3	Goal-directed navigation and architectural refactoring	14
2.5.4	Interactive and developer-in-the-loop recommendation	14
2.5.5	Beyond structural metrics: alternative objectives for ranking refactorings	14
2.5.6	Synthesis: what sequence-generation work enables, and what remains missing	15
2.6	Process-aware work and IDE trace capture	15
2.6.1	Logging developer interactions in IDEs	15
2.6.2	Granularity, segmentation, and semantic interpretation	16
2.7	Summary and positioning: toward process-quality evaluation	16
3	Project Plan and Timetable (Jan–Jun 2026)	17
4	Evaluation Plan	18
5	Ethical Issues and Data Handling Plan	19
A	First Appendix	20

List of Figures

1.1	Illustration of refactoring trajectories scored by process-quality metric J . The solid line shows the developer's observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.	7
-----	--	---

List of Tables

2.1	Candidate signals for constructing a process-quality objective. The thesis combines endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.	11
-----	---	----

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development: developers restructure code to improve readability, maintainability, and evolvability while keeping externally observable behaviour the same. Traditional definitions emphasise this preservation of behaviour and describe refactoring as a sequence of small, semantics-preserving transformations [1, 2, 3], and subsequent surveys categorise these refactoring operations and the tools that support them [4]. In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle when enhancing and maintaining code, and long-lived systems need continual adjustment (as argued in work building on Lehman’s laws and empirical studies of software maintenance). At the same time, refactoring can be costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated “refactoring-only” tasks [5, 6].

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common approach is to measure refactoring success as a change in static quality indicators between a “before” and “after” state: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related metrics to do with maintainability. Typical strands of work include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric changes to separate refactoring-like edits from functional ones, or to choose between different refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance by incorporating traceability alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers’ motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to determine developers’ motivations from commit artefacts). Alongside this, there is a substantial line of work that has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), which has enabled large-scale empirical studies of how often refactoring happens, what kinds occur, and in what contexts [7, 8, 9].

However, these perspectives largely evaluate refactoring as a mapping from an initial program state to a final program state. They do not directly address whether the *trajectory* taken to reach the end state was efficient, low-risk, or “good” under any explicit criteria. This matters because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes take detours, redo work, and often accept temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving at a final outcome. Two developers can arrive at the same end state via very different paths: one might take lots of disruptive steps, bounce between designs, or create avoidable churn, while another might get there via a steadier, more stable sequence. Existing endpoint-only evaluation cannot distinguish these different cases. Meanwhile, research on process realism suggests that refactoring is often “flossed” ongoing development instead of being a separate dedicated task, which increases the chance of mixed-intent steps and noisy

traces [5]. This makes the quality of the process (how a developer navigates intermediate states) potentially important for both developer productivity and the health of the codebase as it evolves.

1.3 Research Objectives

This project investigates the following thesis question: **given a known start state and a known end state, can we evaluate the quality of the developer’s refactoring process, and identify points where a better process was available?** The core idea is to explicitly model refactoring as a *process* over states, rather than judging it only by comparing endpoints. To do this, the project adopts a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits—including IDE refactorings and manual transformations—are treated as *actions* that produce a trace from the start state to the end state.

A core component will be a **process-quality metric** that scores traces according to criteria that will be defined and justified using existing research on code quality metrics, smell detection validity, and process-level signals. The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and “safety” proxies such as preserving build and test behaviour, reflecting the known limitations of relying on any one signal alone.

To go beyond scoring a single observed refactoring trace, the project will construct *counterfactual* alternatives in the form of other **reference trajectories** between the same start and end states which have been optimised to achieve a higher score under a chosen process-quality metric. This provides a reasonable basis for comparison: analysis can identify *divergence points* where the developer’s trajectory deviates from a higher-scoring path, and quantify how much of the gap in resulting refactoring quality is due to each divergence. This extends previous work that synthesises or recommends refactoring sequences to reach a target design. For example, ReSynth synthesises sequences of IDE refactorings consistent with a developer’s partial edits to complete a transformation at the code-level from a start state to an end state [10], while Refactoring Navigator suggests refactoring steps to guide a system toward a desired target at an architectural level [11]. Search-based refactoring work further demonstrates that refactoring sequences can be treated as an optimisation problem under explicit (often multi-objective) quality functions [12, 13]. While these systems show that refactoring sequences can be generated and optimised, they do not aim to assess the quality of a developer’s chosen process under an explicit process-quality objective. In this project, counterfactual generation supports *process evaluation*: it is used to construct a higher-scoring reference trajectory under the chosen metric and to explain key divergence points, rather than to automate the developer’s work or to find any arbitrary sequence of steps that can be followed to reach the end state.

1.4 Approach and Scope

The approach proposed in this thesis targets refactoring as it actually occurs in real-world development, including sequences that involve both IDE-supported refactorings and manually performed edits. It initially focuses on refactorings in a single language (e.g., Java) to allow the use of mature parsing infrastructure, established quality metrics, and state-of-the-art refactoring mining (e.g., RefactoringMiner). A refactoring process is represented as a sequence of steps between code snapshots. Steps may correspond to commits or developer-defined checkpoints, and this representation can be refined using higher granularity IDE data where available.

The final system will comprise of (i) an IDE plugin for capturing refactoring traces and presenting feedback, and (ii) an analysis component that scores the observed refactoring process and finds higher-scoring alternative trajectories between the same start and end states under a chosen process-quality metric. Rather than attempting all possible refactoring operators in a given state, or to calculate globally optimal refactoring trajectories, the goal is to produce a plausible, reference trajectory which scores higher on the chosen metric, and to explain the points at which the developer’s trajectory diverged from it (e.g., where making a different earlier decision could have avoided churn, reduced temporary quality degradation, or preserved build/test stability).

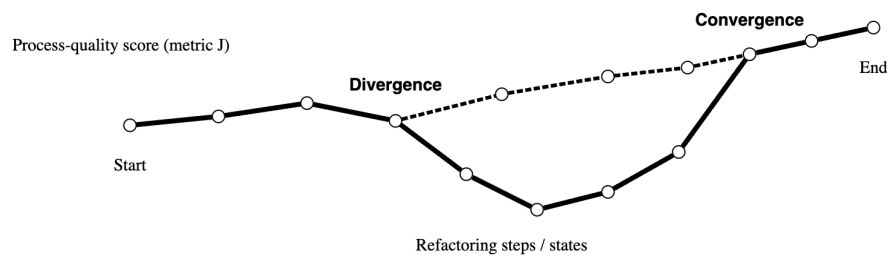


Figure 1.1: Illustration of refactoring trajectories scored by process-quality metric J . The solid line shows the developer's observed trajectory, and the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.

Chapter 2

Background and Related Work

This chapter reviews the background needed to evaluate *refactoring as a process* rather than just refactoring outcomes. It starts with established definitions of refactoring and the concept of refactoring sequences (§2.1). It then covers the more common outcome-based approach for evaluating refactoring (§2.2), before discussing the quality signals typically used to define “better code”, including code smells and static metrics (§2.3). Finally, it looks at methods for detecting and mining refactorings from code changes (§2.4), which is what makes it possible to extract actions and traces from real development histories.

2.1 Definitions and framing

2.1.1 Refactoring as behaviour-preserving transformation

Refactoring is defined as a change to a program’s internal structure intended to improve design qualities (e.g. readability, maintainability) while preserving externally observable behaviour. Most definitions emphasise that refactorings are applied as small, semantics-preserving steps and that safety is ensured through maintaining preconditions and/or validation (e.g. testing) [1, 2, 3]. Opdyke’s foundational work explains refactorings as program transformations equipped with applicability conditions (preconditions) intended to preserve behaviour under a static approximation [1]. Fowler’s catalogue helped popularise refactoring as an incremental activity performed by composing many small operations, grounded in developer-facing mechanics and motivations [2, 3]. Mens and Tourwé later survey the research landscape, bringing together definitions, taxonomies, and tool support, and emphasising the role of precondition checking and program analysis in automated refactoring [4].

In practice, the intent of maintaining behaviour during refactoring is often complicated by the realities of real-world development: refactorings are interleaved with feature and bug-fix work, may be partially performed, and may be mixed with behaviour-changing edits. Consequently, whether a change is “pure refactoring” is often ambiguous when viewed only through version history. This motivates treating behaviour preservation as an *intended property* of refactoring steps, while acknowledging that empirical traces may contain mixed-intent transitions.

2.1.2 Atomic refactorings, smells, and sequences

Research on refactoring often defines a set of *operators* (also described as atomic refactorings) such as EXTRACT METHOD, MOVE METHOD, or RENAME that can be composed together into larger transformations [2, 4]. These operators provide (i) a definitive set of actions for tools and (ii) a way to reason about refactoring sequences. The notion of *code smells* is also closely related: these are heuristic indicators that suggest potential design problems and thus potential refactoring opportunities [2, 3] in codebases. While smells are not formal criteria for correctness, they provide a widely-used bridge between developer intuition and measurable signals (discussed further in §2.3).

A key distinction of this thesis is between evaluating refactoring as an *endpoint mapping* (before vs after) and evaluating refactoring as a *trajectory* (a sequence of intermediate states). Even if two trajectories share the same start and end states, they may differ substantially in intermediate

degradation, churn, or disruption, which can be consequential for productivity and risk during the refactoring process.

2.1.3 State–action–trace formalism

To explicitly represent refactoring as a process, this thesis adopts a state/action/trace framing aligned with the view of refactoring as sequential operator application [2, 4]. Let a codebase snapshot be a *state* $s \in \mathcal{S}$ (e.g. the repository at a particular point in time). Let an *action* $a \in \mathcal{A}$ denote a developer-performed edit step, which may be an IDE-supported refactoring operation, a manually performed transformation, or a mixed of the two. A refactoring process is then a *trace*

$$\tau = (s_0, a_0, s_1, a_1, \dots, a_{T-1}, s_T),$$

where each a_t transforms state s_t into s_{t+1} . Under this framing, “process quality” can be treated as a property of τ (and potentially its prefixes), rather than a property only of (s_0, s_T) .

Two practical issues arise from this. First, the granularity of steps (what constitutes one a_t) is not constant: steps may be commits, IDE events, or developer-defined checkpoints. Second, action labels may be partially observed or inferred (e.g. via refactoring detection tools). These motivate our review of refactoring mining (§2.4) and shape how we design process-quality metrics (§2.3).

2.2 Outcome-based refactoring evaluation

2.2.1 Endpoint-centric quality improvement

A common way to evaluate refactoring is by comparing quality indicators computed on a “before” snapshot and an “after” snapshot. In this framing, refactoring is deemed successful if it improves a chosen set of quality proxies (e.g. reduced coupling, increased cohesion, reduced complexity, fewer code smells). This endpoint-focused perspective appears in both empirical studies of refactoring impact and in automated recommendation systems that rank refactoring candidates by their predicted improvement.

A representative example is *search-based refactoring*, which frames refactoring as an optimisation problem over sequences or sets of refactoring operations. Harman and Tratt demonstrate multi-objective search at the design level, explicitly treating coupling and cohesion as competing objectives and producing Pareto-optimal trade-offs [12]. Later work extends this to many-objective settings and introduces constraints or preferences that reflect practical concerns such as test coverage or prioritisation [13]. These approaches make it explicit that “better” is inherently multi-dimensional, and collapsing everything into a single quality score can be overly restrictive.

This same outcome-based ranking also appears in recommender systems that evaluate alternative refactoring solutions. For example, some systems extend beyond structure-based metrics by incorporating additional information such as requirements traceability, arguing that designs can be “better” not only in properties derivable from the code structure, but also in maintainability tasks like impact analysis [14]. More recently, researchers have revisited the problem of selecting among large sets of refactoring candidates and investigated which code characteristics tend to correlate with positive or negative impacts in practice [15].

2.2.2 What endpoint evaluation captures—and what it misses

Endpoint-based evaluation is best-suited to answer questions of the form: “is the end state better than the start state under metric Q ?” It provides a simple basis for automated recommendation (simulate each candidate and then choose the one that maximises ΔQ) and for empirical analysis (associate refactoring events with changes in quality proxies). It also aligns with the fact that many quality metrics are defined as functions of a single codebase snapshot.

The downside is that it ignores the refactoring process itself, and therefore cannot distinguish between trajectories that reach the same endpoint. In real refactoring, intermediate states can temporarily degrade in ways that matter: code may become harder to understand, modular structure may be disrupted, or build/test stability may regress before it improves again. Furthermore, the *cost* of reaching an improved endpoint is not solely captured by the endpoint metrics alone: disruptive changes can increase churn, complicate review, and introduce integration risk even if final static metrics improve. Empirical work on modularity improvement highlights this trade-off

between optimisation and disruption, showing that metric gains can come with substantial structural disturbance [16]. This motivates evaluating not only final quality, but also the *trajectory cost* and stability of the path taken.

These limitations are central to the thesis: the goal is not to replace outcome-based quality metrics, but to use them within a process-level evaluation that can treat intermediate degradation, rework, churn, and stability as first-class terms in a process quality objective.

2.3 Code smell detection and quality metrics

2.3.1 Smell detection as operationalising design issues

Code smells are a common way to link between qualitative design concerns and measurable indicators [2]. Smell detectors typically detect these smells via rules or learned models. They usually rely on structural properties (e.g. size, coupling, cohesion) and, increasingly, lexical and semantic features also. A prominent rule-based system is DECOR, which defines smells and design anti-patterns using a set of measurable symptoms and corresponding detection rules [17]. DECOR is widely used in empirical studies because its definitions are explicit and it can be executed efficiently at scale.

Despite being useful, smell detectors are not ground truth. Comparative studies often show disagreement between tools in both precision and recall, and that performance is highly variable across different systems and smell types [18]. These differences are largely derived from how each tool defines smells in practice: thresholds, feature sets, and implementation details. Furthermore, detectors may produce false positives: code sections flagged as “smelly” when there is no clear negative impact or that reflect acceptable design trade-offs [19]. For process evaluation, these issues imply that smell counts should be treated as noisy signals, potentially requiring smoothing, calibration, or should be combined with other evidence rather than being used as the sole objective.

2.3.2 Quality metrics: structural and readability-oriented signals

Static metrics offer another way to quantify design quality at the granularity of classes, methods, and packages. Classic metric families quantify coupling, cohesion, complexity, and size, and are widely used as proxies for maintainability. Recent work has also argued for using readability-oriented signals that capture aspects not well-modelled by purely structural measures. For example, readability models that combine structural features (e.g. line structure) with textual features (such as identifier patterns) tend to align better with human judgements of readability [20]. These signals are particularly relevant for evaluating refactoring because readability and understandability are common motivations for refactoring and are often affected by intermediate states.

2.3.3 Implications for process-quality metric construction

The process-quality metric $J(\tau)$ used in this thesis will draw on these signals, but it must also account for their limitations. The literature points to three key design principles:

1. **Use multiple signals.** No single metric or smell detector provides reliable ground truth across contexts [18, 19]. Combining complementary signals can reduce sensitivity to tool-specific noise.
2. **Differentiate endpoint improvement from trajectory cost.** Outcome-based improvements remain valuable, but disruption and intermediate degradation can be significant even when the final snapshot improves [16].
3. **Prefer interpretable components.** For a process critique to be actionable, the metric should be decomposable into terms that can be explained (e.g., readability dip, churn spike, smell increase), rather than a single opaque score.

A further practical implication is that process evaluation should include an explicit notion of developer **effort**, such as the length of the trajectory (number of steps) and/or the magnitude of edits required between successive states. While fewer steps are not inherently better (e.g., small “micro-steps” may reduce risk), step count and edit effort provide a natural regulariser against unnecessary detours and rework, complementing quality-improvement terms and measures

of disruption. Cost terms like these are commonly treated as optimisation objectives alongside structural quality goals in search-based refactoring and recommendation settings (e.g., minimising change while improving quality), and are therefore suitable candidates for trajectory-level scoring [12, 13].

Table 2.1 summarises candidate signals, their typical benefits and limitations, and examples of available tooling in the Java ecosystem. The purpose of the table is not to fully specify the final form of the process-quality metric $J(\tau)$ (e.g., precise term weights and normalisation), but to motivate the choice of component signals and the design principles that the final metric will comprise of. A definition of $J(\tau)$ [will be defined] in the methodology chapters and used consistently throughout the thesis evaluation.

Signal	Pros	Cons / threats	Typical tooling
Structural metrics (coupling/cohesion/complexity/size)	Widely used; efficient; interpretable at class/method granularity	Proxy validity depends on context; may incentivise disruptive restructuring; metric interactions	Static analysis metric extractors; IDE/CI integrations
Smell counts / severity (rule-based)	Directly targets design issues; aligns with developer’s motivations	Detector disagreement and false positives; threshold sensitivity; context dependence	DECOR-style detectors [17]; other smell tools [18]
Readability models	Closer to human comprehension than structural-only metrics; captures lexical/style clues	Model generalisation and dataset bias; may be language/style dependent	Readability predictors [20]
Churn / disruption (LOC changed, file moves/renames)	Captures cost and instability of change; naturally trajectory-oriented	Churn may be necessary for large refactors; can penalise required restructuring	VCS mining; diff statistics
Process length / effort (number of steps; edit magnitude)	Trajectory-native cost signal; discourages detours/rework; easy to compute	Sensitive to step granularity (commits vs. checkpoints vs. IDE events); may penalise safe micro-steps; requires calibration	VCS checkpoints; IDE logs; diff/AST edit size
Build/test preservation proxies	Direct safety signal; aligns with “behaviour-preserving intent”	Requires runnable builds/tests; noisy due to flaky tests; environment dependence	CI logs; local build/test runs
Traceability-augmented signals	Reflects maintenance tasks beyond code structure; complements metric-only ranking	Requires trace links (often unavailable); domain-specific assumptions	Traceability-based ranking methods [14]

Table 2.1: Candidate signals for constructing a process-quality objective. The thesis combines endpoint quality change with trajectory-oriented cost and stability terms, rather than relying on any single indicator.

2.4 Refactoring detection and mining from code changes

2.4.1 From diffs to refactoring actions

To evaluate a process trace in practice, you usually need to either (i) capture developer actions directly (for example, IDE event logs) or (ii) infer actions from the differences between snapshots (like commits). A lot of work focuses on the second case: given two versions of a program, infer whether a refactoring happened and, if it did, identify which refactoring types occurred.

A key technical enabler is *structured code differencing*, usually built on abstract syntax trees (ASTs). This makes it possible to detect moves, renames, and other non-local edits that line-based diffs tend to miss. GumTree is a representative AST differencing approach: it builds fine-grained mappings between AST nodes and produces edit scripts that line up better with the structural changes developers actually intended [9]. This kind of differencing is often used as a base layer for refactoring detection.

Building on these foundations, modern refactoring miners detect refactoring instances by combining structural mappings with refactoring-specific heuristics. RefactoringMiner is a well-known

tool that detects a wide range of refactoring types by analysing successive revisions, using structured matching to relate program entities across versions [8]. RefDiff takes a similar direction: it detects common refactoring types in version histories using similarity heuristics and static analysis [21]. Together, these tools support large-scale mining studies and can also provide an *action vocabulary* for traces, for example by labelling a transition $s_t \rightarrow s_{t+1}$ with a multiset of detected refactoring operations.

2.4.2 Representation, coverage, and typical failure modes

Refactoring detection tools typically output refactoring instances using a taxonomy of operator types (e.g., MOVE METHOD, EXTRACT CLASS) and the program entities involved. This is useful because it matches the operator-based view of refactoring used in catalogues and surveys [2, 4]. However, the literature repeatedly reports several limitations, and these are especially important when doing process evaluation.

Mixed and tangled changes. Version control commits often mix refactoring and functional edits, or they bundle multiple unrelated activities together. In these cases, a commit-level transition can include both behaviour-preserving and behaviour-changing edits, and refactoring detection may only return partial results or it may be ambiguous. This reduces how interpretable the actions extracted from snapshots are, and it motivates careful trace construction (for example, using finer checkpoints) and methods that are robust to noisy labels.

Incomplete coverage and composition effects. Detectors only cover a finite set of operators, and refactorings can also be composed (e.g., move+rename) in ways that make matching harder. Some refactorings are done manually without tool support, and they might not fit the patterns that detectors expect. For process evaluation, this means that “actions” should be treated as only partially observed: detected refactorings are useful as informative features, but they are not an exhaustive description of developer intent.

Behaviour preservation is not verified. Detection is usually structural: it infers that a change *looks like* a refactoring, but it does not prove that behaviour was preserved. Because of this, downstream evaluation should not treat “detected refactoring” as the same thing as a “safe refactoring”. It should also use independent safety proxies where possible (for example, whether the build passes or tests still pass).

2.4.3 Tie-in to this thesis

Refactoring mining is important infrastructure for building traces and for characterising actions at scale [8, 21]. However, the contribution of this thesis is not to improve refactoring detection accuracy. Instead, mined refactorings are used as part of the observational layer: they provide labels and features that describe transitions between states, which can support constructing a process-quality metric and generating reference trajectories. The next part of this background builds on these ideas to discuss prior work on refactoring sequence generation and navigation, which is directly relevant to constructing higher-scoring reference trajectories under a chosen objective.

2.5 Refactoring recommendation and search/synthesis

The previous sections motivate two requirements for process-level refactoring evaluation: (i) a way to represent and score a developer trace, and (ii) a way to construct counterfactual *alternative* trajectories for comparison. A substantial body of work already generates refactoring suggestions and sequences, but it usually targets objectives that are different from evaluating a developer’s process. Broadly, existing approaches fall into three overlapping families: (a) *synthesis* approaches that reconstruct or complete refactoring sequences from examples or partial edits; (b) *search-based* approaches that optimise refactoring sequences under quality objectives; and (c) *interactive* approaches that adapt recommendations based on developer feedback. This section reviews these families and explains how they can support, but do not directly solve, process-quality evaluation.

2.5.1 Synthesis and path completion from partial edits

One way to generate counterfactual alternatives is to treat refactoring as a program transformation problem: given a start program and a target (or partially edited) program, synthesise a sequence of refactoring operators that carries out the transformation. ReSynth is a representative system in this space [10]. It uses an interactive workflow where a developer makes some edits manually and then asks the tool to complete the transformation by synthesising a sequence of IDE-supported refactorings that is consistent with those edits. On the technical side, ReSynth constrains the search by focusing on the entities that were edited, and by searching for operator sequences that transform the original program into a version that matches the developer’s partial edits. This shows two points that matter for this thesis: first, refactoring can be framed as a structured search problem over refactoring operators; and second, the search space can be constrained using evidence from the developer trace itself, rather than naively enumerating all possible refactoring sequences.

A closely related line of work generalises “from examples” synthesis beyond classical refactorings. REFAZER learns program transformations from input-output examples of code edits [22]. Instead of assuming a fixed refactoring catalogue, it synthesises AST-level rewrite rules from a small number of examples and can apply them to automate repetitive edits. While REFAZER is not a refactoring miner in the narrow Fowler sense, it is relevant to counterfactual generation in two ways: (i) it offers a way to infer candidate transformations without predefining an exhaustive operator set; and (ii) it shows that “plausible alternatives” can be derived from edit evidence, which may help bridge the gap between IDE-supported refactorings and manually performed edits.

Relevance and limitations for process evaluation. Synthesis systems mainly aim to *reach* a target program (or a class of targets that are consistent with partial edits) and to do so using sequences that satisfy refactoring preconditions [10]. The objective is reachability and consistency, not evaluating the quality of a developer’s chosen trajectory. In particular, synthesis does not completely answer whether the developer’s intermediate decisions were efficient, low-churn, or risk-minimising under a chosen process objective. Still, synthesis is useful as a source of *candidate alternative trajectories*: it provides a principled way to generate sequences that are likely to be feasible and realistic in a development environment.

2.5.2 Search-based optimisation of refactoring sequences

A second family treats refactoring recommendation as an optimisation problem. Search-based refactoring methods explore sequences or sets of refactorings and then select those that optimise one or more objective functions based on quality metrics [12]. Harman and Tratt argue explicitly for multi-objective formulations (e.g., coupling vs. cohesion) and produce Pareto fronts of trade-offs, rather than forcing design quality into a single scalar [12]. These methods are appealing because they make “better” more measurable, and they can produce many candidate solutions that show the trade-offs more clearly.

Later work extends these formulations to more realistic settings by adding extra objectives. For example, many-objective approaches include not only quality improvements, but also concerns like prioritisation and how refactoring effort is spread across the codebase [13]. These objectives are close to process evaluation because they start to encode costs and constraints that show up during development (e.g., avoiding refactoring that is too concentrated, or respecting priorities), instead of treating refactoring as only a structural optimisation.

Search-based techniques also appear in work that looks at the tension between metric improvement and disruption. Empirical studies of modularity optimisation show that maximising cohesion/coupling improvements can cause substantial disruption compared to the original modular structure [16]. This matters for process evaluation: even if an endpoint configuration is “better” under structural metrics, the path to reach it can be very disruptive, and disruption itself may be a cost term that process evaluation should capture.

Relevance and limitations for process evaluation. Search-based refactoring provides two key ingredients: an explicit objective function and a mechanism to generate candidate sequences. However, most work evaluates the *recommended* refactoring sequence, not a human trace. The algorithm’s output is treated as the solution, and success is measured by the improvement it produces (or by user study preference for the recommendation). This is different from the present

thesis, where the goal is to evaluate a developer’s observed trajectory *relative to* alternative trajectories under an objective that is meant to capture process quality. Still, search-based methods give a natural way to construct a *reference trajectory*: given a start state (and potentially an end constraint), they can produce a plausible trajectory that scores highly under the chosen process objective and can act as a baseline for comparison.

2.5.3 Goal-directed navigation and architectural refactoring

A third family focuses on guiding refactoring toward a target design. Refactoring Navigator is one of these: it takes a current system and a desired target architecture/design, then recommends a sequence of refactoring steps using a search-based strategy, presented through a navigation metaphor [11]. The key idea is to guide developers step-by-step toward a goal, rather than just ranking isolated refactorings. Controlled evaluations report large reductions in task time and manual edits compared to unguided refactoring [11], which suggests that sequence-level guidance can change how developers actually refactor.

Goal-directed refactoring is particularly relevant for the “grand scheme” view of this thesis: it is natural to compare an observed developer trajectory to a higher-scoring “reference” trajectory that reaches a comparable end state. Navigation systems show that it is feasible to produce these stepwise trajectories, and to map high-level goals down to low-level refactoring actions.

Relevance and limitations for process evaluation. Navigation systems typically optimise toward reaching a target design, and they may use design metrics to guide progress [11]. They are not built to evaluate a human’s process quality, and they do not try to explain where a human trace diverged from a better trajectory. However, they provide strong evidence that (i) refactoring can be treated as a sequential decision problem, and (ii) sequence-level suggestions can be computed and presented in ways that developers can actually use.

2.5.4 Interactive and developer-in-the-loop recommendation

Purely automatic optimisation can end up misaligned with developer intent, constraints, or context. Interactive refactoring recommenders address this by incorporating user feedback during the search process. Alizadeh and Kessentini propose an interactive and dynamic search-based approach that presents refactoring recommendations iteratively and updates the search based on developer decisions [23]. This highlights that refactoring recommendation is not only about finding a high-scoring sequence under fixed metrics; it also involves keeping recommendations aligned with developer preferences and constraints as they evolve during a task.

Relevance and limitations for process evaluation. Interactive recommenders are not evaluation tools, but they are closely aligned with the instrumentation and delivery mechanism envisioned by this thesis. If process evaluation is delivered in an IDE, the feedback has to be understandable, minimally disruptive, and sensitive to developer context. Work on interactive recommendation provides ideas for presenting sequence-level suggestions and for building feedback loops into the developer’s workflow [23]. The remaining open problem is to shift from “recommend to optimise” to “compare and evaluate an observed trajectory relative to alternative, counterfactual trajectories”.

2.5.5 Beyond structural metrics: alternative objectives for ranking refactorings

A recurring theme in recommendation research is that structural metrics alone may not be enough to capture what makes a refactoring “good”. One example is ranking refactoring candidates using requirements traceability alongside code metrics. Approaches that include traceability treat “quality” as partly determined by how well requirements map to code, and they are motivated by maintenance tasks like impact analysis and comprehension [14]. This supports the idea that process-quality objectives should not be assumed to be purely structural: the relevant costs and benefits of refactoring can include non-structural properties and context-specific constraints.

Recent work also looks at developer motivations for refactoring, and tries to align recommendations with what developers are likely to prioritise. Studies that mine refactoring operations and repository history investigate product/process signals that correlate with refactoring activity

and with developer intent, including proposals that use LLMs to infer motivations from commit artefacts [24, 25]. While these works mostly focus on *when* and *why* refactoring happens, rather than *how well* it is carried out as a process, they suggest that developer-aligned objectives may need to incorporate process signals and contextual information.

2.5.6 Synthesis: what sequence-generation work enables, and what remains missing

Sequence generation and recommendation systems establish that it is feasible to:

- generate refactoring sequences that realise a transformation or reach a target (synthesis/navigation) [10, 11],
- optimise sequences under explicit metric objectives (search-based refactoring) [12, 13],
- incorporate developer feedback into iterative suggestion mechanisms (interactive recommendation) [23],
- and broaden “quality” beyond structural metrics where additional artefacts exist (e.g., traceability) [14].

However, these systems do not directly answer the two central questions posed by process-level evaluation:

1. *Was the developer’s observed trajectory good under an explicit process-quality criterion?*
2. *Where did the developer’s trajectory diverge from a higher-quality trajectory, and can that divergence be explained?*

The distinction is subtle but important. Recommendation and synthesis work typically treats the algorithm’s output as the primary object and evaluates its outcome or usefulness. Process evaluation instead treats the *developer trace* as the primary object and seeks to compare it against counterfactual trajectories constructed under a process-quality objective.

2.6 Process-aware work and IDE trace capture

Evaluating a refactoring process requires access to traces: sequences of intermediate states and actions. There are two broad sources of traces: (i) version control histories (commits/checkpoints), and (ii) IDE-level interaction logs that capture finer-grained behaviour. This section focuses on IDE logging and process-aware studies, since they show it is feasible to capture refactoring traces at a useful granularity, and they also highlight practical limitations that end up determining evaluation design.

2.6.1 Logging developer interactions in IDEs

Multiple studies have collected large-scale logs of developer interactions to mine workflows and behavioural patterns. Damevski et al. present an approach for mining sequences of developer interactions in Visual Studio using telemetry-like event streams, with the aim of discovering frequent patterns of usage without necessarily recording the full code content [26]. This is relevant for trace collection because it shows that useful traces can be captured as structured events (e.g., commands, navigation actions, edits) while still maintaining the developer’s privacy.

Poncin et al. use Eclipse plugins to collect detailed event logs and then apply process mining techniques to infer developer workflows [27]. This gives a clear example of turning low-level interaction events into higher-level activity models, such as sequences of edits, navigation, compilation, and tool invocations. It also shows some of the practical challenges: event streams are noisy, developers multitask, and mapping raw events to semantically meaningful steps needs careful segmentation and interpretation.

2.6.2 Granularity, segmentation, and semantic interpretation

IDE logging raises the question of what counts as a “step” in a refactoring process. At commit granularity, steps are coarse and often tangled with non-refactoring changes. At event granularity, steps are extremely fine and need to be aggregated into coherent actions. The process mining and interaction analysis literature highlights that segmenting activity into tasks/episodes is not trivial [?, ?]. For refactoring in particular, a single conceptual refactoring can involve several micro-actions (navigate, edit, run a refactoring tool, adjust code, run tests). On the other hand, a short burst of edits might correspond to multiple actions.

A process-evaluation system therefore has to choose a step definition that is both (i) practical to extract reliably and (ii) meaningful enough to allow for interpretation.

2.7 Summary and positioning: toward process-quality evaluation

The literature discussed in §2.1–§2.6 establishes a strong foundations for modelling and measuring refactoring, but it also leaves open a clear space for process-level evaluation.

First, refactoring is well-defined as behaviour-preserving and supported by catalogues of operators and by tool-based precondition checking [1, 2, 4]. Second, most evaluation work is still focused primarily on endpoints: it assesses refactoring success by looking at changes in quality indicators between start and end snapshots, including structural metrics and smell-based proxies [12, 17]. These signals are useful, but they are imperfect and can disagree across tools, which motivates combining multiple signals and metrics. [18, 19, 20]. Third, refactoring mining and AST differencing provide practical ways to infer refactoring actions from code changes, which enables trace reconstruction from repository histories [8, 9]. Fourth, recommendation and synthesis systems show that it is feasible to generate refactoring sequences and plan toward final goal states [10, 11, 23], and that “quality” may need to include contextual objectives beyond just structural metrics [14, 24, 25]. Finally, IDE logging research shows that fine-grained traces can be captured and analysed, but it also highlights challenges around the segmentation and interpretation of them [27, 26, 28].

While there has been substantial research on *detecting* refactorings, *recommending* refactorings, and *synthesising* refactoring sequences, there is comparatively little work that explicitly evaluates the *quality of a developer’s refactoring trajectory* between a known start and end state. In particular, existing research generally does not define and evaluate a criterion for “good refactoring process” that penalises intermediate degradation, disruption, or instability, nor does it provide a specific method to compare a developer’s refactoring trajectory against alternative, higher-quality trajectories (relative to the chosen metric) and to explain divergence points from those trajectories.

This thesis addresses this gap by treating refactoring as a trajectory optimisation and comparison problem: given a start state, an end state (or end constraint), and an observed trace, defining a process-quality objective and constructing a higher-scoring reference trajectory for comparison. This leads to the overall thesis question: *given a known start state and end state, can we produce actionable explanations of a developer’s refactoring process by constructing a plausible reference trajectory and highlighting where and why the developer’s path diverged?*

Chapter 3

Project Plan and Timetable (Jan–Jun 2026)

Work completed so far (as of 21 Jan 2026).

- Extensive related-work research completed (refactoring definitions, endpoint evaluation, smells/metrics, mining, sequence generation, IDE logging), with an initial BibTeX file in progress.
- Early drafts written for the Introduction and Background/Related Work chapters

Timeline in 5 phases (Jan → end of June).

- **Late Jan–Feb:** define refactoring *episodes* and system outputs (trace format, step definition, required explanations), pick an initial metric (*Metric v0*), and finalise the IDE plugin plan.
- **Feb–Mar:** implement the IDE plugin and begin collecting a first batch of refactoring episodes/traces using the plugin.
- **Mar–Apr:** build the offline analysis pipeline (episode → feature extraction → scoring) and implement baseline explanation outputs (per-step breakdown and “what got worse/why” style diagnostics).
- **Apr–May:** implement reference-trajectory generation (v1) and divergence detection (developer trajectory vs. reference trajectory), producing explanations of where/why the paths differ.
- **May–Jun:** run evaluation (human traces + simulated “unclean” variants, with optional open-source mining if feasible), consolidate results, and complete the final report write-up and polishing.

Chapter 4

Evaluation Plan

The main thing I need to show is that the system can take a real refactoring trace and produce explanations that a developer would actually find useful. Concretely, given (i) a repository, (ii) a chosen start and end revision, and (iii) an observed trace of intermediate snapshots, the system should (a) score the observed trajectory, (b) construct a plausible higher-scoring reference trajectory between the same endpoints, and (c) highlight a small set of divergence points with short “what changed / why it matters / what could have been done instead” explanations.

My primary plan is to collect refactoring episodes from humans using my custom IDE plugin, so I have traces in exactly the format the system is designed for. These episodes should include both IDE-supported refactorings and manual edits, and capture whatever signals I can log reliably (e.g., checkpoints/snapshots, refactoring commands, and build/test outcomes where available). This gives me realistic traces with the right granularity, without relying on commit history as the only source of “steps”.

On top of these real episodes, I will create controlled variants by deliberately injecting known bad-process patterns (e.g., unnecessary detours, rework, churn spikes, temporary quality regressions, or broken intermediate builds/tests). The point is to create a rough ground truth for where the divergence *should* be. I can then check whether the system finds these injected divergence points and whether the constructed reference trajectory avoids the injected detours while still reaching the same end state. I will measure this mainly by whether the reference reduces the “bad” signals I injected (e.g., effort/churn, temporary smell/metric spikes, intermediate build/test failures), rather than only by endpoint quality.

I am also considering mining additional episodes from open-source Java repositories, especially small and relatively self-contained refactoring PRs or multi-commit refactoring sequences, but I am not committing to that yet. If I do it, the goal would be to stress-test robustness to real-world messiness (mixed edits, inconsistent commit granularity, incomplete separation between refactoring and feature work) and to increase the diversity of codebases beyond what I can collect through the plugin alone.

I will also run a small human evaluation. The plan is to show participants a refactoring episode plus the system’s divergence explanations, and compare this against a few simple baselines: (i) endpoint-only before/after metrics, (ii) process plots that just flag the largest per-step regressions (no reference trajectory), and (iii) a weak “greedy/local” baseline that makes only local one-step improvements. Participants will rate which output better helps them understand what went wrong in the process and what they would do differently, and give a usefulness/actionability score. I will also collect short free-text comments so I can see what kinds of explanations felt convincing vs. annoying.

Chapter 5

Ethical Issues and Data Handling Plan

The main ethical considerations in this project come from collecting developer traces via an IDE plugin, and from the possibility that the tool could be misused as a form of monitoring. The technical work itself is not safety-critical, but the data involved can still be sensitive because it relates to how people work and may include source code artefacts.

If I collect refactoring episodes from human participants, I will use informed consent and make it clear what is logged (e.g., refactoring commands, checkpoints/snapshots, basic metadata like timestamps, and build/test outcomes) and what is *not* logged (e.g., personal accounts, unrelated browsing, or continuous keystroke-level recordings unless genuinely required). I will follow a data minimisation approach: only collect the smallest amount of information needed to reconstruct refactoring steps and compute the evaluation metrics. Participants will be able to withdraw, and I will delete their data if requested.

Source code can be commercially sensitive, even when the research goal is about refactoring rather than the code itself. To avoid IP issues, I plan to run participant studies on toy projects and/or open-source repositories, rather than proprietary code. If open-source repositories are used, I will respect licensing requirements and avoid re-publishing large code excerpts in the dissertation. If I later want to evaluate on any non-public codebase, I would only do so with explicit written permission and with clear constraints on what is stored and shared.

Even without names, traces can sometimes be identifying (e.g., file paths, commit messages, timestamps, distinctive code structure). For any stored traces, I will remove or hash direct identifiers (names, usernames, machine paths, repository URLs) and avoid publishing raw logs. In the dissertation, I will present results in aggregated form where possible (e.g., across episodes), and I will be careful not to “name and shame” individual developers or projects when discussing low-quality trajectories.

A tool that evaluates a developer’s “process quality” could be repurposed for workplace surveillance or performance management. Although that is not the intended use, it is important to acknowledge the risk. I will frame the tool as a developer support and coaching system (actionable feedback about code/process), not as a ranking system for people. In particular, I will avoid outputs that resemble employee scorecards or leaderboards, and I will focus on episode-level technical explanations rather than person-level judgements.

All collected data (traces and any derived metrics) will be stored securely with access limited to the project work. I will keep raw logs only for as long as needed to run experiments and compute results, and then retain only the derived, anonymised summaries required for analysis and write-up. If any data needs to be shared with a supervisor or included in an appendix, it will be anonymised and minimised to reduce the chance of leaking sensitive information.

Appendix A

First Appendix

Bibliography

- [1] William F. Opdyke. *Refactoring Object-Oriented Frameworks*. PhD thesis, University of Illinois at Urbana-Champaign, Urbana-Champaign, IL, USA, 1992.
- [2] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [3] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.
- [4] Tom Mens and Tom Tourwé. A survey of software refactoring. *IEEE Transactions on Software Engineering*, 30(2):126–139, 2004.
- [5] Emerson R. Murphy-Hill, Chris Parnin, and Andrew P. Black. How we refactor, and how we know it. In *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, pages 287–297, 2009.
- [6] Miryung Kim, Danny Cai, and Sunghun Kim. An empirical investigation into the role of API-level refactorings during software evolution. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, pages 151–160, 2011.
- [7] Nikolaos Tsantalis et al. Refactoringminer: An open-source tool for automated refactoring detection. In *Proceedings of the International Conference on Software Engineering (ICSE) (Tool Demo Track)*, 2018.
- [8] Nikolaos Tsantalis et al. Refactoringminer 2.0: A multi-language tool for automated refactoring detection. *IEEE Transactions on Software Engineering*, 2020.
- [9] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, pages 313–324. ACM, 2014.
- [10] Veselin Raychev, Max Schäfer, Manu Sridharan, and Martin Vechev. Refactoring with synthesis. In *Proceedings of the 2013 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 339–354. ACM, 2013.
- [11] Yu Lin, Xin Peng, Yumin Cai, Danny Dig, Li Zhang, and Wenyun Zhao. Interactive and guided architectural refactoring with search-based recommendation. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 535–546. ACM, 2016.
- [12] Mark Harman and Laurence Tratt. Pareto optimal search-based refactoring at the design level. *Journal of Systems and Software*, 80(4):522–534, 2007.
- [13] M. Mohan and Des Greer. Using a many-objective approach to investigate automated refactoring. *Information and Software Technology*, 2019.
- [14] Wang et al. 2018.
- [15] Nikolaos Nikolaidis et al. A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 2024.
- [16] Matheus Paixão et al. An empirical study of cohesion and coupling: Balancing optimisation and disruption. *IEEE Transactions on Evolutionary Computation*, 2017.

- [17] Naouel Moha et al. Decor: A method for the specification and detection of code and design smells. *IEEE Transactions on Software Engineering*, 2010.
- [18] Thiago Paiva et al. On the evaluation of code smells and detection tools. *Journal of Software Engineering Research and Development*, 2017.
- [19] Francesca Arcelli Fontana et al. Antipattern and code smell false positives: Preliminary conceptualization and classification. In *Proceedings of a workshop on code smells / antipatterns*, 2016.
- [20] Simone Scalabrino et al. A comprehensive model for code readability. *Journal of Systems and Software*, 2018.
- [21] Danilo Silva et al. Refdiff: Detecting refactorings in version histories. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2017.
- [22] Reudismam Rolim, Gustavo Soares, Loris D’Antoni, Oleksandr Polozov, Sumit Gulwani, Sherry Ge, and Neha Rungta. Learning syntactic program transformations from examples. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 404–415. IEEE, 2017.
- [23] Vahid Alizadeh, Marouane Kessentini, Ali Ouni, Mohamed Wiem Mkaouer, Mel Ó Cinnéide, and Yuanfang Cai. An interactive and dynamic search-based approach to software refactoring recommendations. *IEEE Transactions on Software Engineering*, 46(9):932–961, 2020.
- [24] Valentina Pantiuchina, Fiorella Zampetti, Simone Scalabrino, et al. Why developers refactor source code: A mining-based study. *ACM Transactions on Software Engineering and Methodology*, 29(4), 2020.
- [25] Germán Robredo, Isabel Gonzales, Yania Crespo, et al. What were you thinking? an llm-driven large-scale study of refactoring motivations in open-source projects, 2025. arXiv:2509.07763.
- [26] Kostadin Damevski, Thomas Fritz, and David Shepherd. Mining sequences of developer interactions in visual studio for usage smells. *IEEE Transactions on Software Engineering*, 43(4):359–371, 2017.
- [27] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Process mining software development workflows. In *Proceedings of the 15th European Conference on Software Maintenance and Reengineering (CSMR)*, pages 379–382. IEEE, 2011.
- [28] Said Foutsekh, Aiko Yamashita, Fabio Petrillo, Mourad Khomh, and Yann-Gaël Guéhéneuc. Developer interaction traces backed by ide screen recordings from think-aloud sessions. In *Proceedings of the 15th International Conference on Mining Software Repositories (MSR) (Data Showcase)*. ACM, 2018. Dataset paper providing IDE interaction traces with synchronized screen recordings.